

Data Structures

Time & Space Complexity Reference

This reference collects the Big-O complexity of core operations for Arrays, HashMaps, and Heaps, then goes further with a handful of less commonly taught data structures — Linked Lists, Stacks/Queues, Binary Search Trees, Tries, and Disjoint Set (Union-Find) — each with a minimal, working Python implementation.

Arrays

Operation	Worst Case
Access	$O(1)$
Search	$O(n)$
Insertion	$O(n)$
Deletion	$O(n)$

Elements sit in contiguous memory, so indexing is instant, but inserting or deleting in the middle requires shifting every element after it.

HashMaps

Operation	Avg Case
Access	$O(1)$
Search	$O(1)$
Insertion	$O(1)$
Deletion	$O(1)$

A hash function maps keys directly to buckets, giving constant-time operations on average. Worst case degrades to $O(n)$ if many keys collide into the same bucket.

HashSets

Operation	Avg Case
-----------	----------

Add	$O(1)$
Remove	$O(1)$
Contains	$O(1)$
Iteration	$O(n)$

A HashSet is essentially a HashMap that only stores keys (no values) — same underlying hash table, same average-case $O(1)$ for add/remove/contains. It's the go-to structure for deduplication and fast membership tests. Worst case is $O(n)$ per operation if many keys collide into the same bucket.

```
# Python's built-in set IS a hash set
seen = set()
seen.add(4)
seen.add(7)
print(7 in seen)      #  $O(1)$  membership test -> True
seen.discard(4)       #  $O(1)$  removal, no error if missing

# Classic use: fast de-duplication
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)    # {1, 2, 3, 4},  $O(n)$  total

# Classic use: detect duplicates while scanning,  $O(n)$ 
def has_duplicate(nums):
    seen = set()
    for n in nums:
        if n in seen:
            return True
        seen.add(n)
    return False
```

Heaps

Operation	Worst Case
Peek	$O(1)$
Heapify	$O(n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$

A binary heap is a complete binary tree stored in an array. Peeking at the min/max is instant; insertion and deletion only need to "bubble" an element up or down the height of the tree, $\log n$.

Bonus: Other Data Structures Worth Knowing

A quick complexity comparison of a few structures that don't get as much airtime as arrays, hashmaps, and heaps — followed by short Python implementations.

Structure / Operation	Typical Case
Linked List — Access	$O(n)$
Linked List — Insert/Delete (at head)	$O(1)$
Stack / Queue — Push, Pop, Peek	$O(1)$
Binary Search Tree — Search/Insert/Delete (balanced)	$O(\log n)$
Binary Search Tree — Search/Insert/Delete (worst, unbalanced)	$O(n)$
Trie — Search/Insert (word of length k)	$O(k)$
Disjoint Set (Union-Find) — Find/Union (with optimizations)	$O(\alpha(n)) \approx O(1)$

Spotlight: The Trie (Prefix Tree)

One of the least-known but genuinely useful structures. A trie stores strings character by character along tree branches, so words sharing a prefix share the same path. It powers autocomplete, spell-checkers, and IP routing tables. Search and insert cost $O(k)$, where k is the length of the word — independent of how many words are stored.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find(prefix) is not None
```

```

def _find(self, s):
    node = self.root
    for ch in s:
        if ch not in node.children:
            return None
        node = node.children[ch]
    return node

# Usage
trie = Trie()
for w in ["cat", "car", "card", "care"]:
    trie.insert(w)

print(trie.search("car"))          # True
print(trie.search("ca"))          # False (not a full word)
print(trie.starts_with("ca"))     # True

```

Spotlight: Disjoint Set (Union-Find)

Tracks a collection of non-overlapping sets and answers "are these two items in the same group?" almost instantly. It's the engine behind Kruskal's minimum spanning tree algorithm, cycle detection, and network connectivity checks. With path compression and union by rank, each operation runs in near-constant time.

```

class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True

# Usage
ds = DisjointSet(6)
ds.union(0, 1)
ds.union(1, 2)
print(ds.find(0) == ds.find(2)) # True, same set
print(ds.find(0) == ds.find(5)) # False, different set

```

Quick Reference: Linked List & Stack/Queue

```

class Node:
    def __init__(self, val, nxt=None):

```

```
        self.val = val
        self.next = nxt

class LinkedList:
    def __init__(self):
        self.head = None

    def push_front(self, val):          # O(1)
        self.head = Node(val, self.head)

    def to_list(self):                 # O(n)
        out, node = [], self.head
        while node:
            out.append(node.val)
            node = node.next
        return out

# Stack and Queue are usually built on a deque, not a list,
# because deque gives O(1) operations on BOTH ends.
from collections import deque
stack = deque()
stack.append(1); stack.append(2); stack.pop()      # LIFO, O(1)

queue = deque()
queue.append(1); queue.append(2); queue.popleft()  # FIFO, O(1)
```

Generated as a quick-reference companion to the Arrays / HashMaps / Heaps slides.